AIMERZ.ai
Aim.Act.Achieve

NOTES

Introduction to NumPy

Introduction to NumPy

Introduction to NumPy and Its Importance in Data Science

NumPy, short for Numerical Python, is a library for numerical computations, which makes it essential in data science for handling and manipulating data efficiently. Python lists are slower and less memory-efficient because they are heterogeneous. In contrast, NumPy arrays are homogeneous and designed for fast computations.

Practical Importance

- **Speed:** NumPy is optimized for numerical calculations and much faster than lists.
- **Convenience:** With NumPy, performing mathematical operations on large datasets becomes simple and fast.
- **Applications:** Commonly used in data manipulation, scientific computations, machine learning, and more.

Difference Between NumPy Arrays and Python Lists

Feature	Python List	NumPy Array
Type consistency	Heterogeneous	Homogeneous
Performance	Slower (pure Python)	Faster (C-optimized)
Memory usage	Higher	Lower
Operations	Element-by-element (explicit)	Vectorized (implicit)

```
pip install numpy
```

```
import numpy as np

# Check the version of NumPy
print("NumPy Version:", np.__version__)
```

```
NumPy Version: 1.26.4
```

NumPy Arrays: Creating, Accessing, and Modifying

A NumPy array is a grid of values, all of the same data type, which is efficient for data manipulation. It supports multi-dimensional arrays (1D, 2D, and beyond) and allows you to create arrays in various ways (e.g., converting lists, using functions).

Creating an Array

Consider a small dataset representing information about a group of people, with columns for their age, height, and weight.

```
import numpy as np

# Sample dataset as a 2D array
data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])
print("Data Array:\n", data)
```

```
Data Array:
 [[ 25 175  70]
 [ 30 160  60]
 [ 35 180  90]
 [ 40 170  80]
 [ 28 165  68]]
```

The array consists of 5 rows and 3 columns. Each inner list represents a row in the array. The elements in each row could represent different features or attributes. Here, each row represents a person, and each column represents:

- Age (in years)
- Height (in centimeters)
- Weight (in kilograms)

Given the data structure above, you can interpret the rows as follows:

- Row 1: A person aged 25 years, with a height of 175 cm and a weight of 70 kg.
- Row 2: A person aged 30 years, with a height of 160 cm and a weight of 60 kg.
- Row 3: A person aged 35 years, with a height of 180 cm and a weight of 90 kg.
- Row 4: A person aged 40 years, with a height of 170 cm and a weight of 80 kg.
- Row 5: A person aged 28 years, with a height of 165 cm and a weight of 68 kg.

Accessing Elements

```
import numpy as np

# Sample dataset as a 2D array
data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])
print("Data Array:\n", data)

# Accessing age and weight of the second person
age_second_person = data[1, 0]  # row 1, column 0
weight_second_person = data[1, 2]  # row 1, column 2

print("Age of the second person:", age_second_person)
print("Weight of the second person:", weight_second_person)
```

```
Age of the second person: 30
Weight of the second person: 60
```

The code demonstrates how to access specific elements in a 2D NumPy array using indexing. The dataset contains rows representing individuals and columns representing attributes (Age, Height, Weight)

Accessing Age of the Second Person

- The element at row index 1 and column index 0 (data[1, 0]) corresponds to the age of the second person in the dataset.
- Value Accessed: 30 (Age of the second person)

Accessing Weight of the Second Person

- The element at row index 1 and column index 2 (data[1, 2]) corresponds to the weight of the second person in the dataset.
- Value Accessed: 60 (Weight of the second person)

Modifying Elements

```
# Changing the height of the first person to 180
data[0, 1] = 180
print("Modified Data Array:\n", data)
```

```
Modified Data Array:
 [[ 25 180  70]
 [ 30 160  60]
 [ 35 180  90]
 [ 40 170  80]
 [ 28 165  68]]
```

The code shows how to modify elements in a 2D NumPy array by assigning new values using indexing.

Changing the Height of the First Person

- The element at row index 0 and column index 1 (data[0, 1]) corresponds to the height of the first person.
- This value is updated from 175 to 180.

Array Operations: Indexing, Slicing, and Iterating

Indexing and slicing help to retrieve and manipulate specific portions of an array, which is useful when working with specific parts of data. NumPy's array slicing allows you to grab rows, columns, or any subset of data quickly.

Indexing: Retrieving Specific Elements

Indexing is used to access individual elements in a NumPy array by specifying their row and column positions. It provides precise control over which element to retrieve.

Example: To retrieve the height of the second person

```
import numpy as np

# Sample dataset as a 2D array
data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

height_second_person = data[1, 1]
print("Height of the second person:", height_second_person)
```

```
Height of the second person: 160
```

data[1, 1]:

- The first index 1 specifies the row (Person 2).
- The second index 1 specifies the column (Height).

Output: 160, which is the height of the second person.

Indexing is ideal for retrieving specific elements when you know their exact positions.

Slicing: Extracting Rows, Columns, or Subarrays

Slicing allows you to retrieve larger portions of an array, such as entire rows, columns, or subarrays. It uses a range of indices instead of single positions.

Example 1: Extracting a Column

To extract the 'Height' column (index 1)

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

heights = data[:, 1]
print("Heights:", heights)
```

```
Heights: [175 160 180 170 165]
```

data[:, 1]:

- **:** selects all rows.
- **1** specifies the column (Height).

Output: [175, 160, 180, 170, 165] which are the heights of all individuals.

Example 2: Extracting Specific Rows

To extract the ages of the first three people.

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])
```

```
ages_first_three = data[:3, 0]
print("Ages of the first three people:", ages_first_three)
```

```
Ages of the first three people: [25 30 35]
```

data[:3, 0]:

- **:3** selects rows 0 to 2 (first three rows).
- **0** specifies the column (Age).

Output: [25, 30, 35] which are the ages of the first three individuals.

Iterating Over Rows in a 2D NumPy Array

Iteration in NumPy allows you to process each row of a 2D array one at a time. This is useful when you want to examine or manipulate data row by row.

How Iteration Works

The for loop iterates over the array data, where each iteration retrieves one row. Each row represents a single data entry (e.g., a person's attributes in the dataset).

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

# Iterate over each row
for person in data:
    print("Person data:", person)
```

```
Person data: [ 25 175  70]
Person data: [ 30 160  60]
Person data: [ 35 180  90]
Person data: [ 40 170  80]
Person data: [ 28 165  68]
```

- **for person in data:** The for loop accesses each row (1D array) in the dataset.
- During each iteration, the variable person holds one row of data, representing the attributes of one individual.
- **print("Person data:", person)** outputs the entire row, making it easy to analyze the data for each person.

Practical Use Cases

Row-by-Row Analysis: Analyze or manipulate the attributes of each individual.

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

for person in data:
    age, height, weight = person  # Unpack the attributes
    print(f"Age: {age}, Height: {height}, Weight: {weight}")
```

```
Age: 25, Height: 175, Weight: 70
Age: 30, Height: 160, Weight: 60
Age: 35, Height: 180, Weight: 90
Age: 40, Height: 170, Weight: 80
Age: 28, Height: 165, Weight: 68
```

Calculations on Rows: Perform operations like finding the Body Mass Index (BMI) for each person

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

for person in data:
    age, height, weight = person
    bmi = weight / ((height / 100) ** 2)  # BMI formula
    print(f"Person with Age {age} has BMI: {bmi:.2f}")
```

```
Person with Age 25 has BMI: 22.86
Person with Age 30 has BMI: 23.44
Person with Age 35 has BMI: 27.78
Person with Age 40 has BMI: 27.68
Person with Age 28 has BMI: 24.98
```

Array Attributes in NumPy

Understanding array properties like shape, size, and dimensions is essential for effective manipulation and understanding of NumPy arrays.

Array Shape

ndarray.shape: The shape of an array refers to its dimensions and is represented as a tuple (rows, columns). Returns a tuple representing the dimensions (shape) of the array.

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

print("Shape of the array:", data.shape)
```

```
Shape of the array: (5, 3)
```

- The dataset has 5 rows (5 individuals) and 3 columns (Age, Height, Weight).
- Shape (5, 3) indicates this structure.

Array Size

ndarray.size: The size of an array refers to the total number of elements in the array. Returns the total number of elements in the array

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

print("Size of the array:", data.size)
```

```
Size of the array: 15
```

The array contains 5 rows × 3 columns = 15 elements.

Array Dimensions (ndim)

ndarray.ndim: The number of dimensions (or axes) of an array tells us whether it's a 1D, 2D, or higher-dimensional array. Returns the number of dimensions (axes) of the array.

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

print("Number of dimensions:", data.ndim)
```

```
Number of dimensions: 2
```

The dataset is a 2D array because it has rows and columns (Age, Height, and Weight for each individual).

Data Type (dtype)

ndarray.dtype: The data type of an array indicates the type of elements it contains (e.g., integers, floats, etc.). Returns the number of dimensions (axes) of the array.

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

print("Data type of the array:", data.dtype)
```

```
Data type of the array: int64
```

- The elements in the dataset are integers (e.g., ages, heights, and weights).
- NumPy automatically assigned the data type as int64 because all values are integers.

Mathematical Operations on Arrays: Broadcasting and Vectorized Operations

In NumPy, mathematical operations can be performed directly on arrays, offering high performance and efficiency. These operations are powered by vectorization and broadcasting, which eliminate the need for explicit loops and allow seamless handling of arrays with different shapes.

Vectorized Operations

Vectorized operations enable mathematical computations directly on arrays. These operations are applied element-wise and are highly optimized, ensuring better performance compared to using Python loops.

- **Direct Operations**: You can perform operations (like addition, multiplication, division) directly on arrays.
- **Efficiency:** Operations are applied simultaneously to all elements in the array, leveraging optimized C-based implementations under the hood.

Example: Converting Heights from cm to Meters

To convert the 'Height' column in the dataset from centimeters to meters:

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

# Convert heights from cm to meters
heights_in_meters = data[:, 1] / 100
print("Heights in meters:", heights_in_meters)
```

```
Heights in meters: [1.75 1.6  1.8  1.7  1.65]
```

- data[:, 1]: Extracts the second column (Height).
- / 100: Divides each height value by 100 to convert it to meters.

Benefits of Vectorized Operations:

- **Readable Code:** Operations are applied in a single line, making the code cleaner and easier to understand.
- **Performance:** Faster execution, especially for large datasets.

Broadcasting

Broadcasting enables operations on arrays of different shapes by "expanding" the smaller array to match the dimensions of the larger array.

- Automatic Expansion: NumPy automatically aligns the dimensions of arrays so they can operate together.
- Element-wise Application: The smaller array is conceptually repeated across the larger array.

Example: Subtracting 5 Years from Ages

To reduce each person's age by 5 years:

```
import numpy as np

data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

# Subtract 5 years from each person's age
ages_minus_five = data[:, 0] - 5
print("Ages after subtracting 5 years:", ages_minus_five)
```

```
Ages after subtracting 5 years: [20 25 30 35 23]
```

data[:, 0]: Extracts the first column (Age).

-5: Subtracts 5 from each value in the 'Age' column.

Key Features of Broadcasting:

- **No Explicit Loops:** The operation is applied element-wise without needing a loop.
- **Dimensional Compatibility:** Allows operations even when array shapes differ.

Reshaping and Transposing Arrays in NumPy

NumPy provides powerful methods for restructuring and reorganizing array data. Two key operations, reshaping and transposing, allow you to change the structure and orientation of arrays, making them indispensable for data analysis, matrix computations, and machine learning tasks.

Reshaping Arrays

Reshaping changes the shape (dimensions) of an array while preserving its data. Commonly used when reorganizing data for model input, preparing matrices for computations, or aligning data dimensions. The total number of elements in the array must remain the same.

Example: Reshaping an Array

```
import numpy as np

# Sample dataset
data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

# Reshape data array to a 3x5 array
reshaped_data = data.reshape(3, 5)
print("Reshaped Array (3x5):\n", reshaped_data)
```

```
Reshaped Array (3x5):
 [[ 25 175  70  30 160]
 [ 60  35 180  90  40]
 [170  80  28 165  68]]
```

- data.reshape(3, 5): The original array is reshaped into 3 rows and 5 columns.
- The total number of elements remains 15 (since 5x3 = 3x5).

Benefits of Reshaping:

- **Flexibility:** Aligns data dimensions for compatibility in calculations.
- **Reorganization:** Prepares data for specific tasks like machine learning input.

Transposing Arrays

Transposing flips an array's rows and columns. Widely used in linear algebra (e.g., matrix multiplication) and data analysis for reorganizing datasets. No new memory allocation; it simply changes the view of the data.

Example: Transposing an Array

```
import numpy as np

# Sample dataset
data = np.array([
    [25, 175, 70],
    [30, 160, 60],
    [35, 180, 90],
    [40, 170, 80],
    [28, 165, 68]
])

# Transpose the data array
transposed_data = data.T
print("Transposed Array:\n", transposed_data)
```

```
Transposed Array:
 [[ 25  30  35  40  28]
 [175 160 180 170 165]
 [ 70  60  90  80  68]]
```

- data.T: Flips the array, swapping rows and columns.
- Each row in the original array becomes a column in the transposed array, and vice versa.

Benefits of Transposing:

- **Matrix Computations:** Facilitates operations like dot products or solving linear systems.
- **Data Rearrangement:** Easily switch between column-oriented and row-oriented views of data.

Working with N-dimensional Arrays in NumPy

N-dimensional Arrays

NumPy arrays are not restricted to 1D (vectors) or 2D (matrices); they can be N-dimensional. Useful for representing complex data structures, such as:

- Time-series data
- 3D images
- Volumetric data

Each dimension adds a layer of abstraction for managing data in hierarchical or grid-like structures.

Example: Creating a 3D Array

```
import numpy as np

# Creating a 3D array
data_3d = np.array([
    [[1, 2, 3], [4, 5, 6]],
    [[7, 8, 9], [10, 11, 12]]
])
print("3D Array:\n", data_3d)
```

```
3D Array:
 [[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
```

The array data_3d has three dimensions:

- Dimension 1: Contains two 2x3 matrices.
- Dimension 2: Represents rows in each 2D matrix.
- Dimension 3: Represents columns in each row.

Shape of the Array: (2, 2, 3): 2 matrices, each with 2 rows and 3 columns.

Practical Applications:

- 3D Images: Store pixel values in three dimensions (width, height, color channels).
- Scientific Data: Represent time-series data across multiple locations.

Creating Arrays

NumPy makes it easy to create arrays, such as arrays of zeros or ones for initialization or arrays with specific sequences.

Creating Arrays of Zeros: np.zeros()

The np.zeros() function creates an array filled with zeros.

```
import numpy as np

# Create a 3x3 array filled with zeros
zeros_array = np.zeros((3, 3))
print("Zeros Array:\n", zeros_array)
```

```
Zeros Array:
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

Creating Arrays of Ones: np.ones()

The np.ones() function creates an array filled with ones.

```
import numpy as np

# Create a 2x4 array filled with ones
ones_array = np.ones((2, 4))
print("Ones Array:\n", ones_array)
```

```
Ones Array:
 [[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

THANK YOU

AIMERZ.ai
Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan